Recurrent Neural Networks, LSTMs & GRUs

Complete Reference: Theory, Mathematics, Architectures & Backpropagation

RNN · LSTM · GRU · One-to-One · One-to-Many · Many-to-One · Many-to-Many · Loss Functions · BPTT · Use Cases · Key Papers

Contents

1 Foundations of Sequence Modelling **3**
- 1.1 Why Sequences Need Special Architectures . . . 3
- 1.2 Word Embeddings: From Tokens to Vectors . . . 3
- 1.3 Activation Functions Used in RNNs . . . 3
- 1.4 Loss Functions Reference . . . 3

2 Vanilla RNN **4**
- 2.1 Architecture & Intuition . . . 4
- 2.2 Single RNN Cell Diagram . . . 4
- 2.3 Parameter Count . . . 5
- 2.4 Backpropagation Through Time (BPTT) . . . 5
- 2.5 Vanishing & Exploding Gradients . . . 5

3 RNN Sequence Architectures **6**
- 3.1 One-to-One . . . 6
  - 3.1.1 Definition . . . 6
  - 3.1.2 Unrolled Diagram . . . 6
  - 3.1.3 Forward Pass . . . 6
  - 3.1.4 Loss Functions . . . 7
  - 3.1.5 Backpropagation . . . 7
  - 3.1.6 Training vs. Inference . . . 7
  - 3.1.7 Use Cases . . . 7
- 3.2 One-to-Many . . . 7
  - 3.2.1 Definition . . . 7
  - 3.2.2 Unrolled Diagram — Training (Teacher Forcing) . . . 7
  - 3.2.3 Unrolled Diagram — Inference (Autoregressive) . . . 8
  - 3.2.4 Forward Pass Equations . . . 8
  - 3.2.5 Loss Function . . . 8
  - 3.2.6 Backpropagation . . . 8
  - 3.2.7 Teacher Forcing vs. Autoregressive . . . 9
  - 3.2.8 Training & Inference Loops . . . 9
  - 3.2.9 Use Cases . . . 9
- 3.3 Many-to-One . . . 9
  - 3.3.1 Definition . . . 9
  - 3.3.2 Unrolled Diagram — Sentiment Classification . . . 10
  - 3.3.3 Forward Pass . . . 10
  - 3.3.4 Loss Functions . . . 10
  - 3.3.5 Backpropagation (BPTT) . . . 10
  - 3.3.6 Variable-Length Sequences . . . 11

3.3.7 Training vs. Inference . . . 11
3.3.8 Use Cases . . . 11
3.4 Many-to-Many . . . 11
3.4.1 Definition . . . 11
3.4.2 4a: Equal-Length (POS Tagging) . . . 11
3.4.3 4b: Unequal-Length — Encoder–Decoder (Seq2Seq) [5] . . . 12
3.4.4 Use Cases . . . 13

4 Long Short-Term Memory (LSTM) 13
4.1 Motivation . . . 13
4.2 LSTM Cell Diagram . . . 13
4.3 LSTM Equations . . . 13
4.4 Why LSTM Solves Vanishing Gradients . . . 14
4.5 LSTM Parameter Count . . . 14
4.6 LSTM Backpropagation Through Time . . . 14
4.7 LSTM with Sequence Architectures . . . 14

5 Gated Recurrent Unit (GRU) 15
5.1 Motivation . . . 15
5.2 GRU Cell Diagram . . . 15
5.3 GRU Equations . . . 15
5.4 GRU Parameter Count . . . 15
5.5 GRU Backpropagation Through Time . . . 16
5.6 GRU Intuition: Update & Reset Gates . . . 16
5.7 GRU with Sequence Architectures . . . 16

6 Comparison: Vanilla RNN vs. LSTM vs. GRU 17
6.1 When to Use Which . . . 17

7 Sequence Architecture Comparison 18

8 Regularisation & Training Hyperparameters 18
8.1 Key Hyperparameters . . . 18
8.2 Regularisation . . . 18

9 Key Seminal Papers 19

Foundations of Sequence Modelling

Why Sequences Need Special Architectures

A feedforward network maps a fixed-size input to a fixed-size output independently at every call. It has *no memory*: given input x_t it knows nothing about x_{t-1}. Many real-world tasks are inherently sequential:

- Natural language: "bank" means something different after "river" vs "money".
- Time series: tomorrow's price depends on today's trend.
- Speech: each phoneme depends on surrounding context.
- Video: each frame carries information from previous frames.

Recurrent networks address this by maintaining a **hidden state** h_t—a summary of everything seen up to time t—that is passed from step to step.

Word Embeddings: From Tokens to Vectors

Every discrete token (word, character, subword) must be converted to a continuous vector before it can enter a neural network.

Let $|V|$ be the vocabulary size and d_e the embedding dimension. Define a learnable matrix $E \in \mathbb{R}^{|V| \times d_e}$.

$$x_t = E[w_t, :] \in \mathbb{R}^{d_e} \tag{1}$$

where $w_t \in \{0, 1, \ldots, |V| - 1\}$ is the integer index of the token at step t. **Embeddings are inputs—not outputs—and are not directly used in the loss.** The matrix E is trained jointly with the rest of the network via backpropagation.

Activation Functions Used in RNNs

Name	Formula	Range	Used for
Tanh	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	Hidden states
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	$(0, 1)$	Gates, binary output
Softmax	$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$	$(0, 1)^K$	Multi-class output
ReLU	$\max(0, z)$	$[0, \infty)$	Occasionally hidden

Loss Functions Reference

Mean Squared Error (regression):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2 \tag{2}$$

Binary Cross-Entropy (2 classes):

$$\mathcal{L}_{\text{BCE}} = -\big[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\big] \tag{3}$$

Categorical Cross-Entropy (K classes):

$$\mathcal{L}_{\text{CE}} = -\sum_{k=1}^{K} y_k \log \hat{y}_k \tag{4}$$

Sequence loss (sum over T steps):

$$\mathcal{L}_{\text{seq}} = \frac{1}{T}\sum_{t=1}^{T} \mathcal{L}_t, \qquad \mathcal{L}_t = -\log \hat{y}_t[y_t^*] \tag{5}$$

Vanilla RNN

Architecture & Intuition

A vanilla RNN processes input sequences one step at a time. At each step t, the **same** set of weights is applied to the current input and the previous hidden state to produce a new hidden state and, optionally, an output.

$$h_t = \tanh\big(W_x\, x_t + W_h\, h_{t-1} + b_h\big) \tag{6}$$

$$z_t = W_y\, h_t + b_y \tag{7}$$

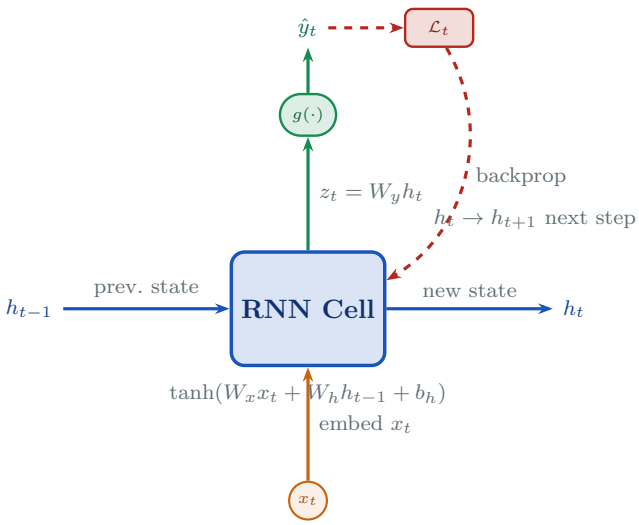
$$\hat{y}_t = g(z_t) \tag{8}$$

Dimensions: $x_t \in \mathbb{R}^{d_e}$, $h_t \in \mathbb{R}^{d_h}$, $W_x \in \mathbb{R}^{d_h \times d_e}$, $W_h \in \mathbb{R}^{d_h \times d_h}$, $W_y \in \mathbb{R}^{d_o \times d_h}$.

$g(\cdot)$: softmax (multi-class), σ (binary), identity (regression).

Critical: W_x, W_h, W_y, b_h, b_y are **shared across all time steps**.

Single RNN Cell Diagram



Parameter Count

Parameter	Shape	#Params
W_x	$d_h \times d_e$	$d_h d_e$
W_h	$d_h \times d_h$	d_h^2
b_h	d_h	d_h
W_y	$d_o \times d_h$	$d_o d_h$
b_y	d_o	d_o
E (embedding)	$\lvert V\rvert \times d_e$	$\lvert V\rvert d_e$
Total		$d_h(d_e + d_h + 1) + d_o(d_h + 1) + \lvert V\rvert d_e$

Backpropagation Through Time (BPTT)

Given sequence loss $\mathcal{L} = \frac{1}{T}\sum_{t=1}^{T} \mathcal{L}_t$, apply the chain rule *unrolled through time*.

Step 1 — Output layer gradient at step t:

$$\delta_t^z = \frac{\partial \mathcal{L}_t}{\partial z_t} = \hat{y}_t - y_t^* \quad \text{(for CE+softmax or BCE+sigmoid)} \tag{9}$$

Step 2 — Error entering hidden state h_t from output:

$$\delta_t^{h,\text{out}} = W_y^\top \delta_t^z \odot (1 - h_t^2) \quad [\text{tanh derivative: } (1 - h_t^2)] \tag{10}$$

Step 3 — Total hidden-state error (from output + from future):

$$\delta_t^h = \delta_t^{h,\text{out}} + W_h^\top \delta_{t+1}^h \odot (1 - h_t^2), \quad \delta_{T+1}^h = \mathbf{0} \tag{11}$$

Step 4 — Weight gradients (accumulated over all steps):

$$\frac{\partial \mathcal{L}}{\partial W_y} = \frac{1}{T}\sum_{t=1}^{T} \delta_t^z h_t^\top \tag{12}$$

$$\frac{\partial \mathcal{L}}{\partial W_h} = \frac{1}{T}\sum_{t=1}^{T} \delta_t^h h_{t-1}^\top \tag{13}$$

$$\frac{\partial \mathcal{L}}{\partial W_x} = \frac{1}{T}\sum_{t=1}^{T} \delta_t^h x_t^\top \tag{14}$$

Step 5 — Embedding gradient (for learnable E):

$$\frac{\partial \mathcal{L}}{\partial E[w_t, :]} = W_x^\top \delta_t^h \tag{15}$$

Unrolled chain (formal):

$$\frac{\partial \mathcal{L}}{\partial W_h} = \sum_{t=1}^{T}\sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial h_t} \underbrace{\left(\prod_{j=k}^{t-1} W_h^\top \operatorname{diag}(1 - h_j^2)\right)}_{\text{product of Jacobians}} h_{k-1}^\top \tag{16}$$

Vanishing & Exploding Gradients

The product of Jacobians in (16) leads to two pathological behaviours:

Let $\lambda_{\max}$ be the largest singular value of $W_h^\top$.

Vanishing ($\lambda_{\max} < 1$):

$$\left\| \prod_{j=k}^{t-1} W_h^\top \operatorname{diag}(1 - h_j^2) \right\| \leq (\lambda_{\max})^{t-k} \to 0 \text{ as } t - k \to \infty$$

Gradient signal from distant past becomes negligibly small ⇒ long-range dependencies not learned.

Exploding ($\lambda_{\max} > 1$): gradient norm grows exponentially ⇒ training diverges.

Fix for exploding — Gradient Clipping [7]:

$$\boldsymbol{g} \leftarrow \boldsymbol{g} \cdot \frac{\theta}{\|\boldsymbol{g}\|} \quad \text{if } \|\boldsymbol{g}\| > \theta, \quad \theta \in [1, 5] \tag{17}$$

Fix for vanishing: use LSTM or GRU (Sections 4–5).

RNN Sequence Architectures

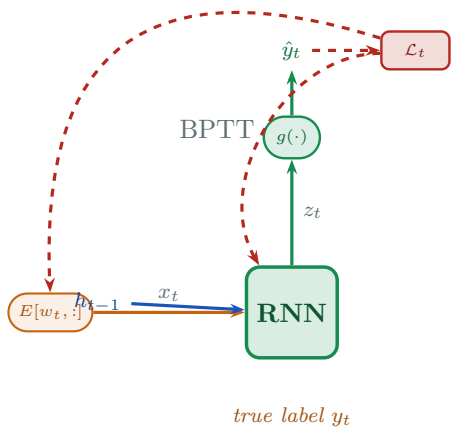
All four architectures below use the **same** RNN cell (eq. 6). The difference lies only in *how many inputs* are fed, *when outputs are produced*, and *how the loss is computed*.

One-to-One

Definition

A single input at one time step produces a single output. Functionally equivalent to a standard feedforward network (MLP) — no sequence dependency is exploited. Sometimes used as the building block in pipelines where sequential context is not needed.

Unrolled Diagram



Forward Pass

$$x_t = E[w_t, :] \tag{18}$$
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h) \tag{19}$$
$$z_t = W_y h_t + b_y \tag{20}$$
$$\hat{y}_t = g(z_t) \tag{21}$$

Loss Functions

Regression: $\mathcal{L} = \frac{1}{2}(\hat{y}_t - y_t)^2$ Binary: $\mathcal{L} = -[y \log \hat{y} + (1-y)\log(1-\hat{y})]$ Multi-class: $\mathcal{L} = -\sum_k y_k \log \hat{y}_k$

Backpropagation

Standard single-step chain rule — no temporal component:

$$\frac{\partial \mathcal{L}}{\partial W_y} = \delta^z h_t^\top, \quad \delta^z = \hat{y}_t - y_t \tag{22}$$

$$\frac{\partial \mathcal{L}}{\partial W_h} = \delta^h h_{t-1}^\top, \quad \delta^h = W_y^\top \delta^z \odot (1 - h_t^2) \tag{23}$$

$$\frac{\partial \mathcal{L}}{\partial W_x} = \delta^h x_t^\top \tag{24}$$

Training vs. Inference

Training: feed x_t; forward pass; compute $\mathcal{L}$; backprop; update weights.
Inference: feed x_t; forward pass; return $\arg\max \hat{y}_t$ or $\hat{y}_t$ directly. No loss computed, no weight update.

Use Cases

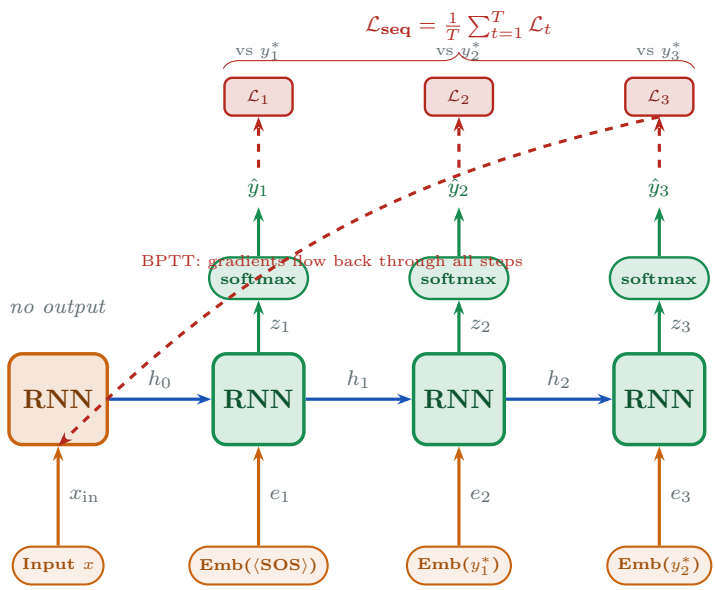
Single time-step anomaly detection; frame-level audio labelling; simple regression tasks where no temporal context is beneficial; pipeline components expecting fixed-size representations.

One-to-Many

Definition

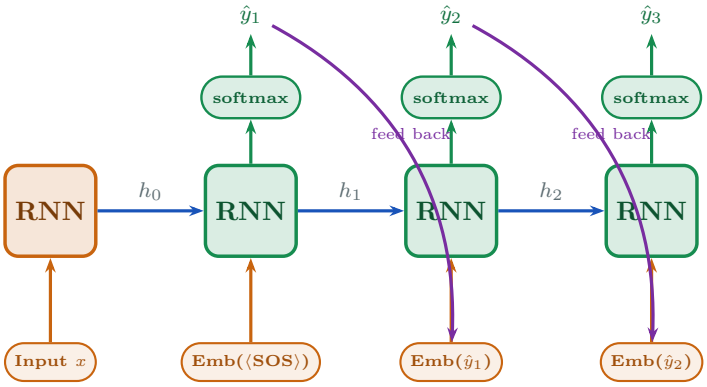
A single input (e.g. an image embedding) seeds the hidden state h_0. The network then auto-regressively generates a sequence of T outputs, one per time step. The only input to subsequent steps is the previous output token (or its ground-truth substitute during teacher forcing).

Unrolled Diagram — Training (Teacher Forcing)



Teacher Forcing: ground-truth y_{t-1}^* fed as input at each step

Unrolled Diagram — Inference (Autoregressive)



Autoregressive Inference: each $\hat{y}_{t-1}$ feeds next step—no ground truth

Forward Pass Equations

Step 0 — seed hidden state:

$$h_0 = \tanh(W_x^{(0)} x_{\text{input}} + b_h) \tag{25}$$

Step $t = 1, \ldots, T$ — generate:

$$e_{t-1} = \begin{cases} E[y^*_{t-1}, :] & \text{teacher forcing (train)} \\ E[\arg\max \hat{y}_{t-1}, :] & \text{autoregressive (infer)} \end{cases} \tag{26}$$

$$h_t = \tanh(W_x e_{t-1} + W_h h_{t-1} + b_h) \tag{27}$$

$$z_t = W_y h_t + b_y \ \in \mathbb{R}^{|V|} \tag{28}$$

$$\hat{y}_t = \text{softmax}(z_t) \tag{29}$$

x_{input} (image / seed) is used **only once**. Its information flows implicitly via $h_0 \to h_1 \to \cdots$

Loss Function

Per-step cross-entropy:

$$\mathcal{L}_t = -\log \hat{y}_t[y^*_t] = -\sum_{k=1}^{|V|} y^*_{t,k} \log \hat{y}_{t,k} \tag{30}$$

Sequence loss:

$$\mathcal{L}_{\text{seq}} = \frac{1}{T} \sum_{t=1}^{T} \mathcal{L}_t \tag{31}$$

Backpropagation

At each step t, hidden state error has *two* sources:

$$\delta^h_t = \underbrace{W_y^\top (\hat{y}_t - y^*_t) \odot (1 - h_t^2)}_{\text{from output at } t} + \underbrace{W_h^\top \delta^h_{t+1} \odot (1 - h_t^2)}_{\text{from step } t+1} \tag{32}$$

Accumulated weight gradients:

$$\frac{\partial \mathcal{L}}{\partial W_h} = \frac{1}{T}\sum_{t=1}^{T} \delta_t^h h_{t-1}^\top, \quad \frac{\partial \mathcal{L}}{\partial W_y} = \frac{1}{T}\sum_{t=1}^{T} (\hat{y}_t - y_t^*) h_t^\top \tag{33}$$

Weights updated **once** per sequence after full backward pass.

Teacher Forcing vs. Autoregressive

Aspect	Teacher Forcing	Autoregressive
Phase	Training	Inference (also optional in training)
Input at step t	True previous token y_{t-1}^*	Predicted token $\hat{y}_{t-1}$
Error compounding	None	Errors cascade
Convergence speed	Fast	Slow / unstable early on
Matches inference?	No (exposure bias)	Yes
Loss computation	Per step, aggregated	Per step, aggregated

Scheduled Sampling [9]: probability p_i of using true token decays over epochs i; interpolates between the two modes.

Training & Inference Loops

1. Encode input $\to h_0$. 2. Feed $\langle$SOS$\rangle$; get $\hat{y}_1$; compute $\mathcal{L}_1$. 3. Feed true y_1^* (teacher forcing); get $\hat{y}_2$; compute $\mathcal{L}_2$. 4. Repeat for T steps. 5. $\mathcal{L}_{\text{seq}} = \frac{1}{T}\sum \mathcal{L}_t$; BPTT through all T steps; update weights once.

1. Encode input $\to h_0$. 2. Feed $\langle$SOS$\rangle$; $\hat{y}_1 = \arg\max \text{softmax}(z_1)$. 3. Feed $E[\hat{y}_1, :]$; predict $\hat{y}_2$. 4. Stop when $\hat{y}_t = \langle$EOS$\rangle$ or max-length reached. No loss, no weight update.

Use Cases

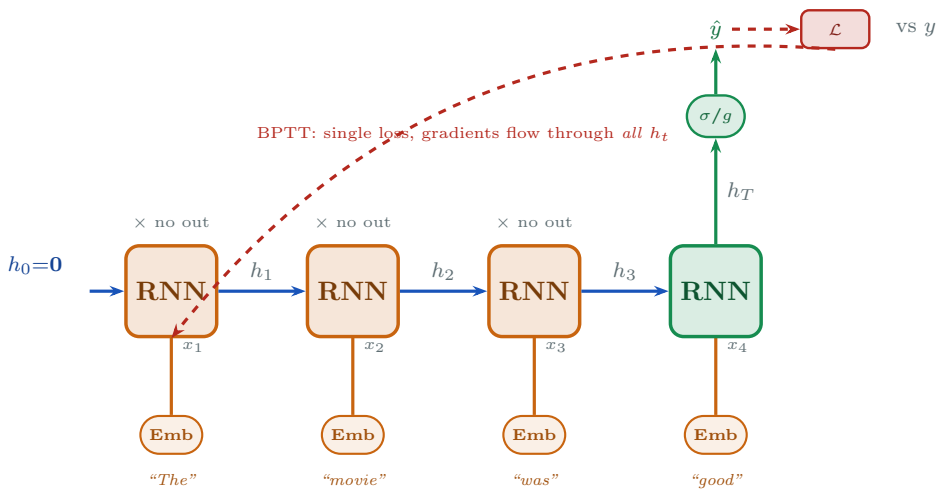
Image captioning [10], music generation, text/code generation from a seed, video description.

Many-to-One

Definition

A sequence of T inputs is processed; **only the final hidden state** h_T produces a single prediction. Hidden states $h_1, \ldots, h_{T-1}$ pass context forward but generate no output. Intuition: "read the entire review, then give one verdict."

Unrolled Diagram — Sentiment Classification



Forward Pass

For $t = 1, \ldots, T$:

$$x_t = E[w_t, :] \tag{34}$$

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h), \quad h_0 = \mathbf{0} \tag{35}$$

Output **only at** $t = T$:

$$z = W_y h_T + b_y \tag{36}$$

$$\hat{y} = g(z) \tag{37}$$

Loss Functions

Binary: $\mathcal{L} = -[y \log \hat{y} + (1-y)\log(1-\hat{y})], \;\; \hat{y} = \sigma(z)$

Multi-class: $\mathcal{L} = -\sum_{k=1}^{K} y_k \log \hat{y}_k, \;\; \hat{y} = \text{softmax}(z)$

Regression: $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2, \;\; \hat{y} = z$

Loss computed **once** — after the sequence is fully processed.

Backpropagation (BPTT)

1. Output gradient: $\delta^z = \hat{y} - y$

2. Error into h_T**:** $\delta_T^h = W_y^\top \delta^z \odot (1 - h_T^2)$

3. Propagate backward $(t = T-1, \ldots, 1)$: $\delta_t^h = W_h^\top \delta_{t+1}^h \odot (1 - h_t^2)$

4. Accumulate: $\frac{\partial \mathcal{L}}{\partial W_h} = \sum_{t=1}^{T} \delta_t^h h_{t-1}^\top, \quad \frac{\partial \mathcal{L}}{\partial W_x} = \sum_{t=1}^{T} \delta_t^h x_t^\top, \quad \frac{\partial \mathcal{L}}{\partial W_y} = \delta^z h_T^\top$

Despite only one loss, gradients flow through **all** T hidden states.

Variable-Length Sequences

In theory: RNN runs for T_i steps for sentence i; outputs h_{T_i}. No padding needed for single-sentence processing.

In batches (practical):

1. Sort by length; pad shorter sequences with ⟨PAD⟩.
2. Use length-masking to extract the *last real* hidden state h_{T_i}, not the padded position.
3. Loss is computed from h_{T_i}, so ⟨PAD⟩ tokens **do not contribute** to the loss.

Training vs. Inference

Training: embed all T words; run forward pass; compute output from h_T; compute loss; BPTT from h_T to h_1; update weights once.

Inference: embed and run T steps; return $\arg\max \hat{y}$ or scalar $\hat{y}$.

Use Cases

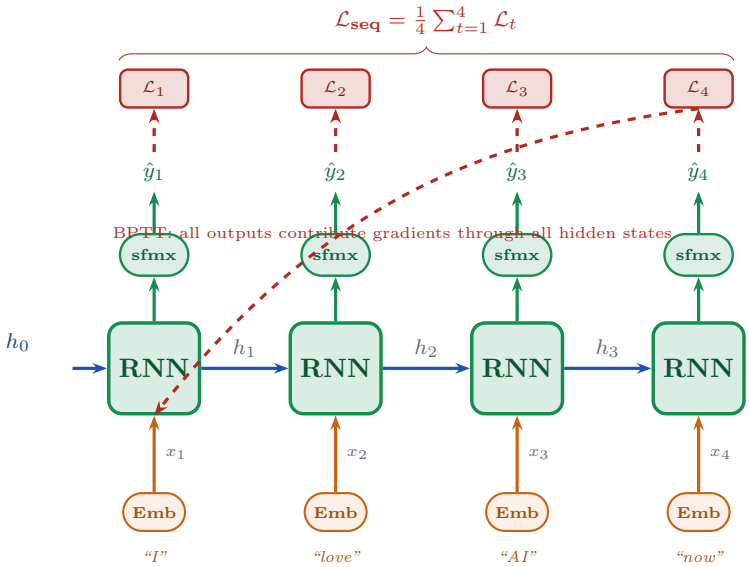
Sentiment analysis [13], spam detection, document classification, stock-trend prediction, speech-emotion recognition, medical time-series diagnosis.

Many-to-Many

Definition

T inputs produce T' outputs ($T' = T$ for equal-length; $T' \neq T$ for Seq2Seq). Every output step contributes to the loss.

4a: Equal-Length (POS Tagging)



Forward Pass:

$$h_t = \tanh(W_x E[w_t, :] + W_h h_{t-1} + b_h), \quad t = 1, \ldots, T \tag{38}$$

$$\hat{y}_t = \text{softmax}(W_y h_t + b_y), \quad t = 1, \ldots, T \tag{39}$$

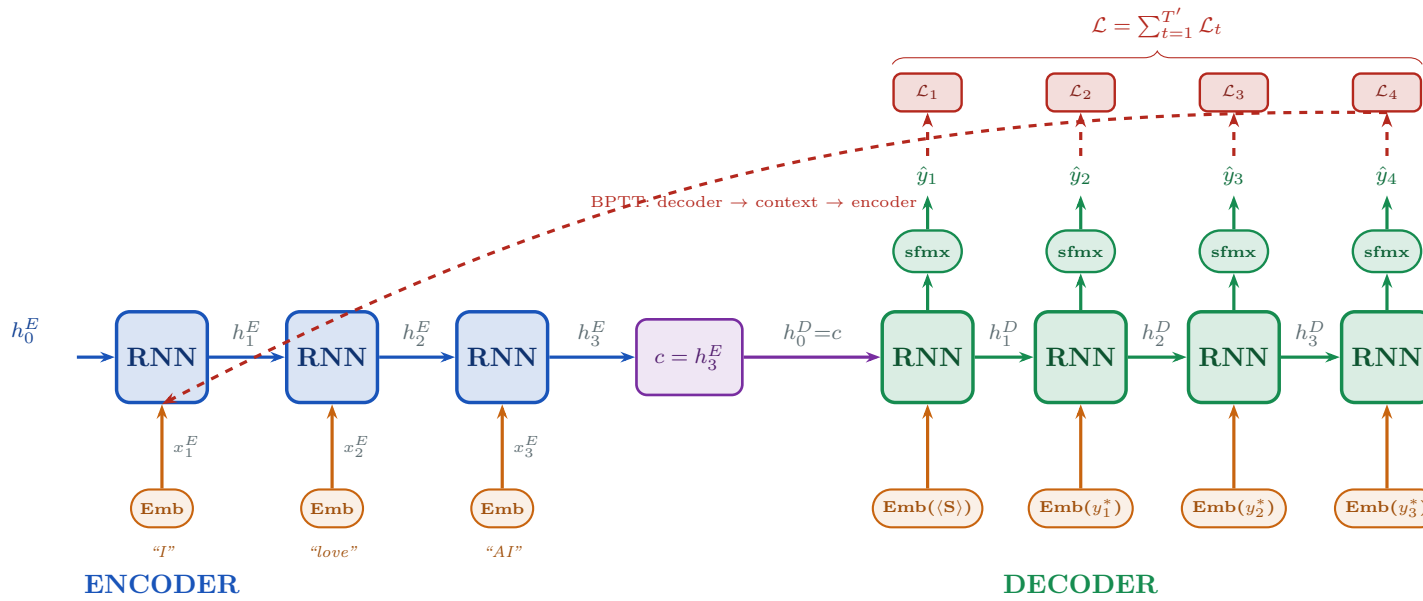
Loss:

$$\mathcal{L}_t = -\log \hat{y}_t[y_t^*], \qquad \mathcal{L}_{\text{seq}} = \frac{1}{T}\sum_{t=1}^{T} \mathcal{L}_t \tag{40}$$

BPTT — hidden error from two sources per step:

$$\delta_t^h = W_y^\top(\hat{y}_t - y_t^*) \odot (1 - h_t^2) + W_h^\top \delta_{t+1}^h \odot (1 - h_t^2), \quad \delta_{T+1}^h = \mathbf{0} \tag{41}$$

4b: Unequal-Length — Encoder–Decoder (Seq2Seq) [5]



Source T=3 $\neq$ Target T'=4

Encoder (source length T):

$$h_t^E = \tanh(W_x^E E^{\text{src}}[w_t^{\text{src}}, :] + W_h^E h_{t-1}^E + b_h^E) \tag{42}$$

$$c = h_T^E \quad \text{(context / thought vector)} \tag{43}$$

Decoder (target length T'):

$$h_0^D = c \tag{44}$$

$$e_{t-1}^D = \begin{cases} E^{\text{tgt}}[y_{t-1}^*, :] & \text{teacher forcing} \\ E^{\text{tgt}}[\hat{y}_{t-1}, :] & \text{autoregressive} \end{cases} \tag{45}$$

$$h_t^D = \tanh(W_h^D h_{t-1}^D + W_e^D e_{t-1}^D + b_h^D) \tag{46}$$

$$\hat{y}_t = \text{softmax}(W_y h_t^D + b_y) \tag{47}$$

Stage 1 — Decoder BPTT:

$$\delta_t^{h,D} = W_y^\top(\hat{y}_t - y_t^*) \odot (1 - {h_t^D}^2) + W_h^{D\top} \delta_{t+1}^{h,D} \odot (1 - {h_t^D}^2) \tag{48}$$

Stage 2 — Gradient into context:

$$\frac{\partial \mathcal{L}}{\partial c} = \delta_1^{h,D} \tag{49}$$

Stage 3 — Encoder BPTT:

$$\delta_T^{h,E} = \frac{\partial \mathcal{L}}{\partial c} \odot (1 - h_T^{E^2}), \quad \delta_t^{h,E} = W_h^{E\top} \delta_{t+1}^{h,E} \odot (1 - h_t^{E^2}) \tag{50}$$

All weights updated in a single backward pass.

Use Cases

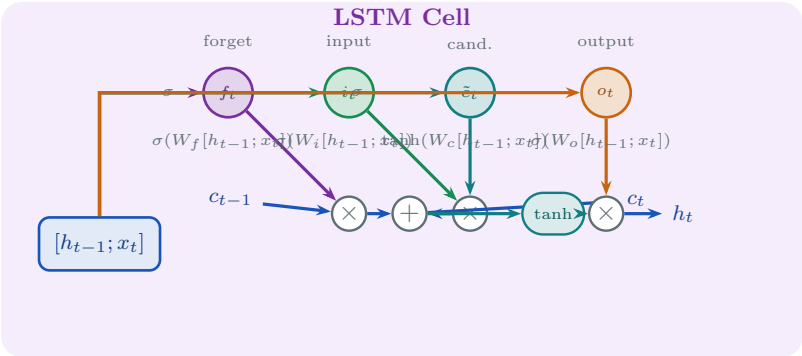
POS tagging / NER (equal), machine translation [5, 6], text summarisation, speech recognition, chatbots.

Long Short-Term Memory (LSTM)

Motivation

Vanilla RNNs suffer from vanishing gradients for long sequences (Section 2.5). Hochreiter & Schmidhuber [2] introduced LSTM cells with **gating mechanisms** and a separate **cell state** c_t that allows gradients to flow over many steps with minimal attenuation.

LSTM Cell Diagram



LSTM Equations

2

Let $[h_{t-1};\, x_t] \in \mathbb{R}^{d_h + d_e}$ be the concatenation of previous hidden state and current input.

$$f_t = \sigma(W_f[h_{t-1};\, x_t] + b_f) \quad \text{(forget gate: what to erase from } c_{t-1}\text{)} \tag{51}$$

$$i_t = \sigma(W_i[h_{t-1};\, x_t] + b_i) \quad \text{(input gate: what new info to write)} \tag{52}$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1};\, x_t] + b_c) \quad \text{(candidate cell state)} \tag{53}$$

$$o_t = \sigma(W_o[h_{t-1};\, x_t] + b_o) \quad \text{(output gate: what to expose)} \tag{54}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad \text{(cell state update)} \tag{55}$$

$$h_t = o_t \odot \tanh(c_t) \quad \text{(hidden state / output)} \tag{56}$$

Dimensions: $W_f, W_i, W_c, W_o \in \mathbb{R}^{d_h \times (d_h + d_e)}$; $b_f, b_i, b_c, b_o \in \mathbb{R}^{d_h}$; $c_t, h_t \in \mathbb{R}^{d_h}$.

Why LSTM Solves Vanishing Gradients

The cell-state update $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ creates an **almost-linear** path for gradients. The gradient of c_t w.r.t. c_{t-1} is simply f_t (elementwise):

$$\frac{\partial c_t}{\partial c_{t-1}} = \mathrm{diag}(f_t) \tag{57}$$

Because the forget gate $f_t \in (0,1)^{d_h}$ is *learned*, the network can set $f_t \approx \mathbf{1}$ for dimensions it wants to remember, keeping gradients close to 1 over many steps. The product of Jacobians for the cell state becomes:

$$\prod_{j=k}^{t-1} \frac{\partial c_{j+1}}{\partial c_j} = \prod_{j=k}^{t-1} \mathrm{diag}(f_{j+1}) \tag{58}$$

which is *controlled* rather than exponentially decaying.

LSTM Parameter Count

Each of the four gate matrices has shape $d_h \times (d_h + d_e)$, giving:

$$\text{LSTM params} = 4\, d_h(d_h + d_e + 1) + d_o(d_h + 1) \tag{59}$$

This is 4× more than a vanilla RNN with the same hidden size.

LSTM Backpropagation Through Time

Define combined input $u_t = [h_{t-1}; x_t]$. Let $\delta_t^h = \frac{\partial \mathcal{L}}{\partial h_t}$, $\delta_t^c = \frac{\partial \mathcal{L}}{\partial c_t}$.

From h_t backward:

$$\delta_t^o = \delta_t^h \odot \tanh(c_t) \tag{60}$$

$$\delta_t^c = \delta_t^h \odot o_t \odot (1 - \tanh^2(c_t)) + \delta_{t+1}^c \odot f_{t+1} \tag{61}$$

From c_t to gates:

$$\delta_t^{\tilde{c}} = \delta_t^c \odot i_t \odot (1 - \tilde{c}_t^2) \tag{62}$$

$$\delta_t^i = \delta_t^c \odot \tilde{c}_t \odot i_t(1 - i_t) \tag{63}$$

$$\delta_t^f = \delta_t^c \odot c_{t-1} \odot f_t(1 - f_t) \tag{64}$$

$$\delta_t^{o,\text{gate}} = \delta_t^o \odot \sigma'(o_t) \tag{65}$$

Gate weight gradients (gate $g \in \{f, i, c, o\}$):

$$\frac{\partial \mathcal{L}}{\partial W_g} = \sum_{t=1}^{T} \delta_t^g\, u_t^\top \tag{66}$$

Back to h_{t-1}:

$$\delta_{t-1}^{h,\text{from above}} = \left(W_f^\top \delta_t^f + W_i^\top \delta_t^i + W_c^\top \delta_t^{\tilde{c}} + W_o^\top \delta_t^{o,\text{gate}}\right)_{[0:d_h]} \tag{67}$$

(Take only the first d_h components—corresponding to h_{t-1}.)

LSTM with Sequence Architectures

LSTM replaces the vanilla RNN cell in all four sequence architectures. The only change is eq. (6) → eqs. (51)–(56). All loss functions, BPTT flow patterns, training/inference loops, and use-case mappings remain **identical**.

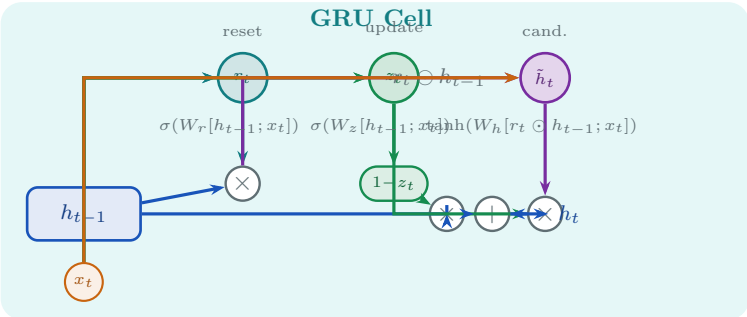
- **Machine translation** [5]: 4-layer LSTM Enc–Dec
- **Language modelling**: character/word-level LMs [12]
- **Speech recognition**: LSTM–CTC models
- **Handwriting recognition**: bidirectional LSTM
- **Question answering**: reading-comprehension models
- **Video captioning**: CNN + LSTM

Gated Recurrent Unit (GRU)

Motivation

Cho et al. [4] proposed GRU as a simpler alternative to LSTM. GRU merges the forget and input gates into a single **update gate** and eliminates the separate cell state, reducing parameters while often matching LSTM performance.

GRU Cell Diagram



GRU Equations

$$z_t = \sigma(W_z[h_{t-1};\, x_t] + b_z) \quad \text{(update gate: blend old vs. new)} \tag{68}$$

$$r_t = \sigma(W_r[h_{t-1};\, x_t] + b_r) \quad \text{(reset gate: how much past to forget)} \tag{69}$$

$$\tilde{h}_t = \tanh\big(W_h[r_t \odot h_{t-1};\, x_t] + b_h\big) \quad \text{(candidate hidden state)} \tag{70}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad \text{(final hidden state)} \tag{71}$$

Dimensions: $W_z, W_r \in \mathbb{R}^{d_h \times (d_h + d_e)}$; $W_h \in \mathbb{R}^{d_h \times (d_h + d_e)}$; $h_t \in \mathbb{R}^{d_h}$ (no separate cell state).

GRU Parameter Count

Three gate matrices (vs. four for LSTM):

$$\text{GRU params} = 3\, d_h(d_h + d_e + 1) + d_o(d_h + 1) \tag{72}$$

≈25% fewer parameters than LSTM with same d_h.

GRU Backpropagation Through Time

Let $\delta_t^h = \frac{\partial \mathcal{L}}{\partial h_t}$.

Step 1 — Gradient through interpolation eq. (71):

$$\frac{\partial \mathcal{L}}{\partial z_t} = \delta_t^h \odot (\tilde{h}_t - h_{t-1}) \odot z_t(1 - z_t) \tag{73}$$

$$\frac{\partial \mathcal{L}}{\partial \tilde{h}_t} = \delta_t^h \odot z_t \odot (1 - \tilde{h}_t^2) \tag{74}$$

Step 2 — Gradient through candidate eq. (70):

$$\frac{\partial \mathcal{L}}{\partial r_t} = \left(W_{h,h}^\top \frac{\partial \mathcal{L}}{\partial \tilde{h}_t} \right) \odot h_{t-1} \odot r_t(1 - r_t) \tag{75}$$

(where $W_{h,h}$ is the h_{t-1}-portion of W_h)
Step 3 — Back to h_{t-1}:

$$\delta_{t-1}^{h,\text{prev}} = \delta_t^h \odot (1 - z_t) + (W_{z,h}^\top \delta_{z,t} + W_{r,h}^\top \delta_{r,t} + W_{h,h}^\top \delta_{\tilde{h},t} \odot r_t) \tag{76}$$

Gate weight gradients:

$$\frac{\partial \mathcal{L}}{\partial W_z} = \sum_t \delta_{z,t}\, u_t^\top, \quad \frac{\partial \mathcal{L}}{\partial W_r} = \sum_t \delta_{r,t}\, u_t^\top, \quad \frac{\partial \mathcal{L}}{\partial W_h} = \sum_t \delta_{\tilde{h},t}\, \tilde{u}_t^\top \tag{77}$$

where $u_t = [h_{t-1}; x_t]$ and $\tilde{u}_t = [r_t \odot h_{t-1}; x_t]$.

GRU Intuition: Update & Reset Gates

- **Update gate** $z_t \approx 1$: $h_t \approx \tilde{h}_t$ — completely replace with new candidate.
  $z_t \approx 0$: $h_t \approx h_{t-1}$ — copy previous hidden state unchanged.
  Controls *how much of the past to carry forward* (analogous to forget+input gates combined in LSTM).
- **Reset gate** $r_t \approx 0$: candidate $\tilde{h}_t$ ignores h_{t-1} — start fresh.
  $r_t \approx 1$: full access to previous state.
  Controls *how much past state influences the candidate.*

When $z_t = 0$, $r_t = 1$: GRU reduces to a vanilla RNN.
When $z_t = 1$: output copies the candidate; memory is fully refreshed.

GRU with Sequence Architectures

GRU drops into all four architectures identically to LSTM. The hidden state h_t (no cell state) is the only recurrent signal.

- Neural machine translation [4]
- Sequence-to-sequence models with fewer parameters
- Real-time/on-device NLP (smaller model footprint)

- Time series with moderate temporal dependencies
- Speech synthesis

Comparison: Vanilla RNN vs. LSTM vs. GRU

	Vanilla RNN	**LSTM**	**GRU**
Cell state	None	c_t (separate)	None (merged into h_t)
Hidden state	h_t	h_t	h_t
Gates	None	f_t, i_t, o_t (3 gates)	z_t, r_t (2 gates)
Parameters	$d_h(d_e + d_h + 1)$	$4d_h(d_e + d_h + 1)$	$3d_h(d_e + d_h + 1)$
Vanishing grad	Severe	Largely solved	Largely solved
Long-range deps	Poor	Excellent	Good–Excellent
Training speed	Fastest	Slowest	Intermediate
Memory usage	Lowest	Highest	Intermediate
Key paper	[1]	[2]	[4]

When to Use Which

- **Short sequences ($T \lesssim 20$), simple tasks:** Vanilla RNN is sufficient; fastest to train.
- **Long sequences, complex dependencies:** Use LSTM or GRU.
- **Limited compute / on-device:** GRU preferred (25% fewer params than LSTM).
- **Maximum accuracy, unlimited compute:** LSTM with multiple layers and dropout.
- **Very long documents / paragraphs:** Consider Transformers, which avoid sequential bottleneck entirely via self-attention [11].

Sequence Architecture Comparison

	One-to-One	**One-to-Many**	**Many-to-One**	**Many-to-Many**
Input	1 step	1 step	T steps	T steps
Output	1 step	T steps	1 step (end)	T or T' steps
Loss	Once	Every output step	Once (final)	Every output step
Loss formula	$\ell(\hat{y}, y)$	$\frac{1}{T}\sum \mathcal{L}_t$	$\ell(\hat{y}, y)$	$\frac{1}{T}\sum \mathcal{L}_t$
BPTT	1 step	All T	All T	All T (or enc+dec)
Teacher forcing?	No	Yes	No	Yes (Seq2Seq)
Typical $g(\cdot)$	σ/sfmx/id	softmax	σ/sfmx/id	softmax
RNN example	Anomaly det.	Image caption	Sentiment	POS tagging
LSTM example	Time series	Music gen.	Doc class.	Machine transl.
GRU example	Sensor pred.	Text gen.	Spam detect.	Summarisation

Regularisation & Training Hyperparameters

Key Hyperparameters

Hyperparameter	**Typical Range**	**Effect**	**Notes**
Hidden size d_h	64–2048	Capacity	Larger = more memory, slower
Embedding dim d_e	50–1024	Word repr.	GloVe/FastText: 100–300
Layers (stacked)	1–4	Depth	Each layer: $h_t^{(l)} = f(h_t^{(l-1)}, h_{t-1}^{(l)})$
Learning rate η	1e-4–1e-2	Update step	Decay on plateau
Clip threshold θ	1–5	Exploding grad.	Always use with RNNs
Dropout	0.1–0.5	Regularisation	See below
Batch size	16–256	Speed/stability	Larger = more stable grad.

Regularisation

Dropout [8]: applied to non-recurrent connections. For recurrent connections, use **variational dropout** (same mask at each step).

L2 regularisation: $\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda \sum_\theta \|\theta\|_2^2$

Early stopping: monitor validation loss; stop when it ceases to improve.

Weight initialisation: orthogonal init for W_h (preserves gradient norms); Xavier/Glorot for W_x, W_y.

Gradient clipping: $\boldsymbol{g} \leftarrow \boldsymbol{g} \cdot \frac{\theta}{\|\boldsymbol{g}\|}$ if $\|\boldsymbol{g}\| > \theta$. [7]

Key Seminal Papers

1. **Vanilla RNN + BPTT:** Rumelhart, Hinton & Williams (1986) [1]. Introduced backpropagation and its application to recurrent networks.
2. **LSTM:** Hochreiter & Schmidhuber (1997) [2]. Long Short-Term Memory; gated cell to solve vanishing gradient.
3. **Vanishing gradient analysis:** Bengio, Simard & Frasconi (1994) [3]. Formal characterisation of vanishing/exploding gradients.
4. **GRU / Seq2Seq:** Cho et al. (2014) [4]. Encoder–decoder with GRU for neural machine translation.
5. **Seq2Seq with LSTM:** Sutskever, Vinyals & Le (2014) [5]. 4-layer LSTM encoder–decoder for translation.
6. **Attention:** Bahdanau, Cho & Bengio (2015) [6]. Neural attention mechanism; solved fixed-bottleneck problem in Seq2Seq.
7. **Gradient clipping:** Pascanu, Mikolov & Bengio (2013) [7]. Theoretical and practical analysis of exploding gradients; clipping fix.
8. **Dropout:** Srivastava et al. (2014) [8]. Dropout regularisation (widely applied to RNNs).
9. **Scheduled Sampling:** Bengio et al. (2015) [9]. Curriculum learning to bridge train–inference gap.
10. **Image Captioning:** Vinyals et al. (2015) [10]. "Show and Tell" — CNN + LSTM for One-to-Many captioning.
11. **Transformers:** Vaswani et al. (2017) [11]. "Attention Is All You Need"; successor to RNN/LSTM for sequences.
12. **LSTM for language modelling:** Zaremba, Sutskever & Vinyals (2014) [12]. Recurrent neural network regularisation.

References

[1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.

[2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[3] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.

[4] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. EMNLP*, pp. 1724–1734, 2014.

[5] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 3104–3112, 2014.

[6] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. ICLR*, 2015.

[7] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proc. ICML*, pp. 1310–1318, 2013.

[8] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[9] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled sampling for sequence prediction with recurrent neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 1171–1179, 2015.

[10] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan, "Show and tell: A neural image caption generator," in *Proc. CVPR*, pp. 3156–3164, 2015.

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Aiki, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 5998–6008, 2017.

[12] W. Zaremba, I. Sutskever, and O. Vinyals, "Recurrent neural network regularization," *arXiv preprint arXiv:1409.2329*, 2014.

[13] D. Tang, B. Qin, and T. Liu, "Document modeling with gated recurrent neural network for sentiment classification," in *Proc. EMNLP*, pp. 1422–1432, 2015.